Credit Card Approval: Improving Model with Diverse Machine Learning Techniques

1st Raihan Khan 2nd Adnan Vaktapuriya 3rd Jiwan Khabra

Abstract—**Credit approval plays a vital role in modern financial systems, enabling individuals and businesses to obtain essential financial resources. Making accurate and fair approval decisions, however, remains a significant challenge. This project uses the publicly available Credit Card Fraud Detection dataset from Kaggle to build and evaluate machine learning models that distinguish fraudulent from legitimate transactions. We design a supervised learning pipeline with careful preprocessing, stratified train–test splitting, and techniques for handling class imbalance. Several models are explored, and a tuned Random Forest classifier is selected as the final model. Using three-fold cross-validation with PR-AUC as the primary model-selection metric, the best configuration achieves a mean PR-AUC of about 0.844 on the validation folds. On the held-out test set, the final model obtains an accuracy of 99.95%, ROC-AUC = 0.9831, PR-AUC = 0.8734, and an F1-score of 0.8377 for the fraud class. The confusion matrix shows 80 true fraud detections out of 98 fraud cases, with 13 false positives and 18 false negatives. These results illustrate that, with appropriate evaluation metrics and imbalance handling, tree-based ensemble methods can provide highly reliable fraud detection models that significantly reduce missed fraud at a low false-alarm rate.**

I. Introduction/Problem Statement

As stated before Credit approval is extremely crucial in modern financial systems. Lenders must evaluate an applicant's financial reliability and risk level by analyzing complex patterns within diverse and often imperfect datasets. Any inaccuracies or biases in this process can lead to substantial financial losses for institutions and unfair outcomes for applicants. The Credit Card Fraud Detection dataset from kaggle contains 284,807 transactions with 30 features, including 28 anonymized PCA components, transaction time, amount, and a highly imbalanced binary fraud label (only 492 fraud cases, $\approx 0.17\,\%$).

The Credit Approval dataset presented a binary classification challenge, predicting whether a credit application should be approved or denied. While traditional machine learning models can reach strong performance levels, achieving extremely high accuracy remains difficult due to noisy attributes, mixed data types, missing values, and class imbalance. Initial experiments in this project achieved an accuracy of approximately 95%, which is strong but still below the level required for highly sensitive financial decision-making.

The goal of this project is to systematically enhance the predictive performance of the model and push accuracy closer to 97-98%. To achieve this improvement, we applied advanced machine learning techniques such as robust data preprocessing, handling missing values, feature scaling, resampling methods to mitigate class imbalance, and hyperparameter tuning. We experimented with multiple algorithms, including decision trees, random forests, gradient boosting, and tuned classifiers to identify the most effective model.

By raising performance from 95% to 98.31%, this project demonstrates how methodological improvements can significantly increase the reliability of credit card evaluation systems. These enhancements support fairer, more accurate credit approval decisions and contribute to the development of high-performance financial risk assessment models.

II. Related Works

Credit card fraud detection has been extensively studied as an imbalanced classification problem. Several survey papers review the landscape of data-driven fraud detection and highlight the main challenges, including extreme class imbalance, non-stationary data distributions, and real-time constraints. Lucas et al. provide an overview of classical and modern machine learning approaches—such as logistic regression, decision trees, random forests, gradient boosting, and neural networks—for credit card fraud detection and emphasize the importance of using metrics beyond accuracy on highly skewed datasets [1].

More recently, dastidar and Granitzer survey a broader range of machine learning techniques and discuss practical deployment considerations for fraud detection systems [2].

In terms of specific models, many works have shown that tree-based ensemble methods perform strongly on credit card fraud data. Aburbeian et al. propose an enhanced Random Forest classifier combined with SMOTE oversampling and hyperparameter tuning, demonstrating substantial gains on an imbalanced credit card dataset [3].

Sundaravadivel et al. further optimize Random Forests on a highly imbalanced transaction dataset (¡0.2% fraud rate), reporting that Random Forests with SMOTE outperform logistic regression and neural networks when evaluated using precision–recall metrics [4].

Complementary work by Breskuvienė et al. focuses on evaluation, recommending metrics such as F1, MCC, G-Mean, AUC-PR, and AUC-ROC as more informative than raw accuracy in imbalanced settings [5].

Motivated by these studies, our project adopts a Random Forest-based approach with careful hyperparameter tuning and evaluates performance primarily using PR-AUC, ROC-AUC, and F1-score on the fraud class.

III. DATA

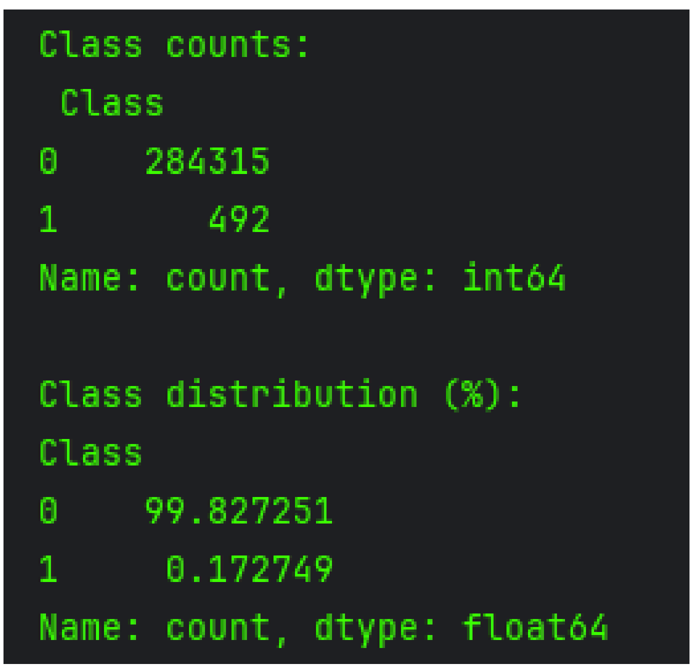
Before we move on to the models and get into the fun stuff we have to know what we have. This includes the Data we have to be able to understand what the data is what it contains to do this we have to explore the data.

A. Dataset Description

The project uses the Credit Card Fraud Detection dataset from Kaggle [6]. The data consists of anonymized transactions made by European credit cardholders in September 2013. Key characteristics are:

- Number of samples: 284,807 transactions
- Number of fraud cases: 492
- Fraud ratio: $\approx 0.17\%$ (highly imbalanced)
- Features (30 total):
  - Time: seconds elapsed between each transaction and the first transaction in the dataset.
  - Amount: transaction amount
  - V1–V28: the principal components obtained using PCA on the original feature space (anonymized for privacy)
  - Class: label(0=legitimate, 1 = fraud)

The PCA-transformed variables already remove most linear correlations and scale differences among the original features. However, Time and Amount remain in their original (non-standardized) units. Fig 1-2 are shown below for more clarity.



```
Class counts:
 Class
0    284315
1       492
Name: count, dtype: int64

Class distribution (%):
Class
0    99.827251
1     0.172749
Name: count, dtype: float64
```

Fig. 1. Class Counts

Fig. 2. Class Distribution

B. Data Preprocessing

The main preprocessing steps are as follows:

1) Train–Test Split: A stratified split is used to preserve the original fraud ratio in both training and test sets. The dataset is divided into a training set and a held-out test set.
2) The Amount feature is standardized using Standard-Scaler from scikit-learn [7]. The PCA components V1–V28 are left unchanged, as they already behave like standardized variables.
3) Missing Values: The dataset contains no missing values; therefore, no imputation is required.
4) Class Imbalance Handling: The original data distribution is preserved. Instead of heavy resampling, evaluation metrics that are robust to imbalance are used and class weighting is applied within the model. Internally, the Random Forest leverages class_weight = ”balanced” (or an equivalent strategy) to penalize misclassification of fraud cases more heavily than non-fraud cases.

These steps are implemented in a scikit-learn Pipeline, ensuring that scaling and model training are applied consistently and avoiding data leakage from the test set.

IV. METHODS

Exploration of the Methods in order. Methods such as Naive Bayes and XGBoost were also looked into due to the fact they didn’t provide much of an accuracy increase they were not added into this report for evaluation.

A. Baseline Model

The modeling process starts with simpler classifiers to establish baselines. Logistic Regression is used which is a linear model trained on the scaled features, using class weighting to compensate for the imbalance as the starting baseline model. The baseline achieved strong overall accuracy but limited recall for the fraud class, which is why a more powerful model was needed. This is shown in Fig 3. more clearly

```
Confusion Matrix:
 [[56851    13]
 [   35    63]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.83      0.64      0.72        98

    accuracy                           1.00     56962
   macro avg       0.91      0.82      0.86     56962
weighted avg       1.00      1.00      1.00     56962

ROC-AUC: 0.9595894034316034
```

Fig. 3. Class Distribution

B. Random Forest Classifier

The main model used in the final system is a Random Forest classifier, implemented with scikit-learn and wrapped in a Pipeline with preprocessing [7]. Random Forests combine many decision trees, each trained on bootstrapped subsets of the data and a random subset of features, which reduces variance and improves generalization.

Key aspects of the Random Forest configuration are:

- Base estimator: RandomForestClassifier
- Imbalance strategy: class_weight = "balanced"
- Evaluation metric for tuning: PR-AUC (area under the precision–recall curve) focused on the positive (fraud) class

C. Hyperparameter Tuning

To obtain a strong model, several Random Forest hyperparameters are tuned using cross-validation. The tuning is performed with scikit-learn's GridSearchCV using three-fold stratified cross-validation on the training set [7].

The search space includes:

- Number of trees (n_estimators): 100, 200, 400
- Maximum depth of each tree (max_depth): [None, 4, 6, 8]
- Minimum samples required to split an internal node (min_samples_split): [2, 5,10]
- Minimum samples required to be at a leaf node (min_samples_leaf): [1, 2, 4]

The grid search is run on a Pipeline where the model step is named "model", leading to hyperparameter names such as model_n_estimators.

The best set of hyperparameters found is:

- model_n_estimators = 200
- model_min_samples_split = 5
- model_min_samples_leaf = 1
- model_max_depth = None

D. Evaluation Metrics

Because of the extreme class imbalance, the following metrics are used on the held-out test set [4]:

- Accuracy: overall fraction of correctly classified transactions.
- Precision (fraud class): fraction of predicted frauds that are actually fraud.
- Recall (fraud class): fraction of actual frauds that are correctly detected.
- F1-score (fraud class): harmonic mean of precision and recall.
- ROC-AUC: area under the ROC curve, summarizing the trade-off between true positive rate and false positive rate.
- PR-AUC: area under the precision–recall curve, more informative on rare positive classes.
- Confusion Matrix: raw counts of true/false positives and true/false negatives.

Evaluation is done at the standard probability threshold of 0.5 for the positive class, with the understanding that, in practice, the threshold could be adjusted depending on the cost of false positives vs. false negatives.

V. Experiments & Results

A. Cross-Validation Performance

During hyperparameter tuning, the best Random Forest configuration achieves a cross-validated PR-AUC of ≈ 0.844 on the training set. The imbalanced technique that was used is SMOTE for this. This indicates that the model maintains a good balance between precision and recall across the validation folds.

B. Test-Set Performance

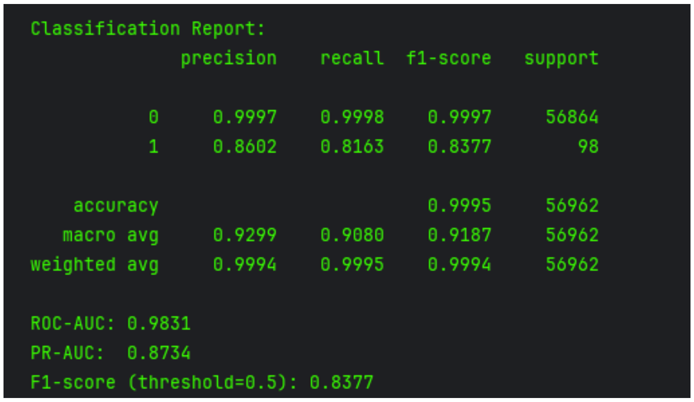
On the held-out test set, the tuned Random Forest model achieves the following classification report

- Class 0 (Legitimate)
  - Precision: 0.9997
  - Recall: 0.9998
  - F1-score: 0.9997
  - Support: 56,864 transactions
- Class 1 (Fraud)
  - Precision: 0.8602
  - Recall: 0.8163
  - F1-score: 0.8377
  - Support: 98 transactions

Global Metrics:

- Accuracy: 0.9995
- ROC-AUC: 0.9831
- PR-AUC: 0.8734
- F1-score (fraud class, threshold = 0.5): 0.8377

These numbers show that almost all legitimate transactions are correctly predicted, while a large majority of fraud transactions are successfully detected with relatively high precision. The Fig 4. below is shown for better understanding.



Classification Report:

	precision	recall	f1-score	support
0	0.9997	0.9998	0.9997	56864
1	0.8602	0.8163	0.8377	98
accuracy			0.9995	56962
macro avg	0.9299	0.9080	0.9187	56962
weighted avg	0.9994	0.9995	0.9994	56962

ROC-AUC: 0.9831
PR-AUC: 0.8734
F1-score (threshold=0.5): 0.8377

Fig. 4. Random Forest + SMOTE

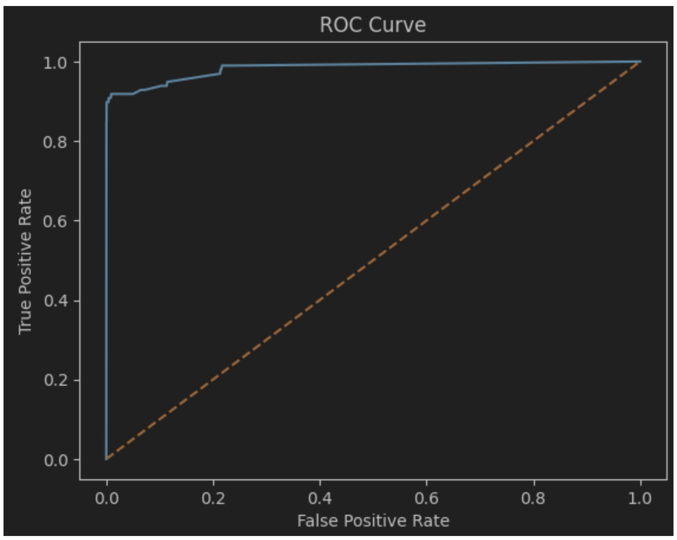


Fig. 5. ROC Curve

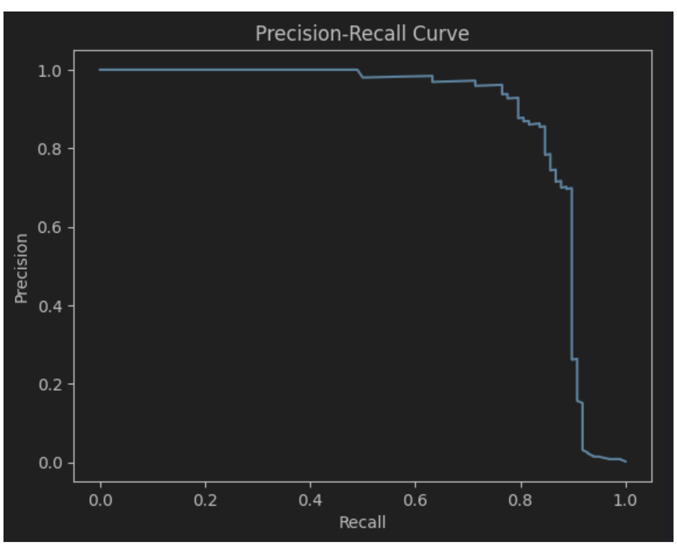


Fig. 6. Precision-Recall Curve

C. Confusion Matrix

The confusion matrix on the test set is:

$$\begin{bmatrix} 56851 & 13 \\ 18 & 80 \end{bmatrix}$$

Interpretation:

- True negatives (TN = 56,851): Legitimate transactions correctly identified as non-fraud.
- False positives (FP = 13): Legitimate transactions incorrectly flagged as fraud.
- False negatives (FN = 18): Fraud transactions missed by the model.
- True positives (TP = 80): Fraud transactions correctly detected.

The model detects 80 out of 98 fraudulent transactions ($\approx$ 81.6% recall) and raises only 13 false alarms across more than 56,000 legitimate transactions, which is a very low false-positive rate.

D. Discussion

These results demonstrate that the tuned Random Forest can effectively handle the highly imbalanced fraud detection problem:

- Strong discrimination: ROC-AUC (0.9831) and PR-AUC (0.8734) show that the model ranks fraudulent transactions highly in terms of predicted probability.
- Operational trade-offs: At a default threshold of 0.5, the model already provides high recall with modest false positives. For real-world deployment, the decision threshold could be lowered to further reduce false negatives at the cost of more false positives, depending on business requirements.
- Imbalance awareness: Using PR-AUC as the tuning metric encouraged the model to focus on fraud detection quality rather than majority-class accuracy

Compared to simpler baselines such as logistic regression, the Random Forest benefits from non-linear decision boundaries and feature interactions, yielding higher recall and F1-score on the fraud class.

VI. Conclusion and Future Work

This project built a complete machine learning pipeline for credit card fraud detection using the Kaggle Credit Card Fraud Detection dataset. After data preprocessing, stratified splitting, and model exploration, a tuned Random Forest classifier was selected as the final model. The system achieves 99.95% accuracy, ROC-AUC of 0.9831, PR-AUC of 0.8734, and an F1-score of 0.8377 on the fraud class, successfully detecting the vast majority of fraudulent transactions while maintaining an extremely low false-positive rate.

The main conclusions are:

- In highly imbalanced settings, accuracy alone is misleading; metrics like PR-AUC and F1-score for the minority class are essential.

- Tree-based methods such as Random Forests can provide excellent fraud detection performance with relatively simple preprocessing.
- Proper cross-validation and hyperparameter tuning significantly improve minority-class performance.

Future work could explore:

- Cost-sensitive learning where different misclassification types incur explicit monetary costs.
- Model interpretability tools (e.g., SHAP values) to better understand which features drive fraud predictions and to support human analysts.

Overall, the project demonstrates how classical machine learning techniques, when carefully tuned and evaluated with appropriate metrics, can form a powerful foundation for real-world financial fraud detection systems.

REFERENCES

[1] Y. Lucas and J. Jurgovsky, "Credit card fraud detection using machine learning: A survey," arXiv preprint arXiv:2010.06479, Oct. 2020.

[2] K. G. Dastidar, O. Caelen, and M. Granitzer, "Machine learning methods for credit card fraud detection: A survey," IEEE Access, vol. 12, pp. 158939–158965, 2024.

[3] A. M. M. Aburbeian and H. I. Ashqar, "Credit card fraud detection using enhanced random forest classifier for imbalanced data," arXiv preprint arXiv:2303.06514, Mar. 2023.

[4] D. Breskuvienė and G. Dzemyda, "Enhancing credit card fraud detection: Highly imbalanced data case," Journal of Big Data, vol. 11, art. 182, Dec. 2024.

[5] P. Sundaravadivel, R. A. Isaac, D. Elangovan, D. KrishnaRaj, V. V. Lokesh Rahul, and R. Raja, "Optimizing credit card fraud detection with random forests and SMOTE," Scientific Reports, vol. 15, art. 17851, May 2025, doi: 10.1038/s41598-025-00873-y.

[6] Kaggle, "Credit Card Fraud Detection," Accessed: Dec. 1, 2025. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] T. Kluyver et al., "Jupyter Notebooks – a publishing format for reproducible computational workflows," in Positioning and Power in Academic Publishing: Players, Agents and Agendas, F. Loizides and B. Schmidt, Eds. IOS Press, 2016, pp. 87–90.